

# STRATA: What Process Mining Contributes to HVAC Fault Detection

Event abstraction, stratified discovery and count channels  
on five public benchmarks, and the measured limit of  
conformance checking on its own

Yassh Singh

Thompson Rivers University, Kamloops, BC, Canada

**Abstract.** We report STRATA, a stratified process-mining framework for building HVAC fault detection, and measure what each of its process-mining parts contributes. Sensor streams become an event log by control-sequence rules under a tested partition that keeps fault signatures out of discovery; Petri nets are discovered at device, unit and cross-system strata; eight channels calibrated on fault-free data score each day. On five public LBNL systems the detector catches 158 of 175 seeded faults at 26 false-alarm days in 480 held out, names the affected terminal unit in 37 of 50 determinable detections and never a wrong one, and loses at most one detection per system under field conditions. The event log carries much of this: a frequency channel read from it catches instability faults no rule or residual sees, and discovery located an invariant an expert then wrote down as a rule. The limitation is stated as one: conformance checking of the discovered nets does not by itself detect the seeded faults. No scenario in 73, nor in 199 over six systems, is detected only by the two conformance channels; removing them changes no scorecard and lowers one system’s false alarms from nine days in 96 to four. A measured circularity bound (1.78% against 5.24% recall), a reversal probe (two of three nets order-blind) and an alignment support that reproduces raw counts exactly explain why. Process mining helps to get detection, but it is not detecting on its own.

**Keywords:** Process mining · Event abstraction · Conformance checking · Fault detection.

## 1 Introduction: an empty quadrant, and what we built in it

Process mining has spread from business processes into cyber-physical and sensor-driven domains, and one domain of that kind is missing. Building heating, ventilation and air-conditioning (HVAC) operation is a continuous-flow, standard-governed, heavily instrumented process, and it does not appear: the one systematic review of process mining on sensor data whose scope could have captured it

records no building HVAC application [5], and a survey of 211 AI-based HVAC fault-detection studies has no process-mining entry [3].

The quadrant looked worth filling for a specific reason. The largest empirical characterisation of HVAC faults, across more than 60,000 pieces of equipment in 317 buildings, catalogues the ten most common air-handler faults [6], and on our reading seven of them are behavioural rather than component-level: schedules at the wrong hours, overrides never released, resets skipped, economiser sequences executed out of order. These are deviations in the ordering of operating states, which the threshold alarms and rule libraries buildings run see only if someone wrote a rule for them in advance. Our hypothesis was the obvious one: a model of the order in which a building moves between states should have an advantage on exactly those faults. We pre-registered it with the observation that would refute it, and built the pipeline to test it.

The pipeline that resulted, STRATA, is a working detector, and its process-mining parts do measurable work. An event abstraction grounded in the control sequence turns sensor streams into a daily log (Section 3); nets discovered at three strata give the log a per-device and per-unit structure (Section 4); and channels that read the log — counts of activities, absence of a scheduled activity on a device — carry detections that no rule or residual carries, and name the terminal unit an alarm belongs to (Section 5). On five public benchmark systems the detector catches 158 of 175 seeded faults at 26 false-alarm days in 480 held out. What it does not do is also measured, and is this paper’s centre: conformance checking against the discovered nets, the part of process mining the hypothesis rested on, detects no fault on its own. The falsifier for the founding hypothesis — that a hand-written rule would out-cover the discovered models on the same scenarios — was written down before the rule existed, and fired. The one precedent we know of is a preprint reporting naïve process mining ineffective for intrusion detection on network logs [18]; ours is accompanied by artefacts from which every number regenerates. The HVAC domain is the setting; the results are about process mining: it helps to get detection, and it is not detecting on its own.

## 2 Related work

*Conformance as an anomaly detector.* Scoring traces against a discovered model to flag anomalous ones is an established line [2], on conformance foundations laid by Rozinat and van der Aalst [13, 1]. The line then moved: neural sequence models trained on event logs are reported to outperform conformance-based baselines on standard corpora [11]. What the neural line has not done, and Section 5 does, is to hold the comparison at the level of the individual channel inside one detector on one dataset, where the question — is the conformance signal ever the thing that catches a fault? — has a yes-or-no answer per scenario.

*Event abstraction and leakage.* Turning sensor streams into activity sequences is the binding step for any sensor-domain application; we follow Janssen and

colleagues [9] within the taxonomy of van Zelst and colleagues [17]. The one decision in that step with a measurable consequence is a leakage decision in the sense of Weytjens and De Weerd [16]: Section 3 measures what a discovered model loses when it may no longer recognise the analyst’s own rules.

*Protocol.* Pitsch and colleagues argue that process-mining research should adopt explicit hypothesis testing [12]; the pre-registration and binding falsifiers behind this paper are one attempt to practise it.

### 3 Event abstraction, the alphabet wall, and a measured circularity bound

#### 3.1 The abstraction

HVAC operation offers no natural case boundary: a building runs continuously and its signals are sampled rather than emitted. We take the calendar day as the case, the shortest unit over which the control sequences of interest (start-up, occupancy, economiser transitions, night cycling, shutdown) complete. Streams become events by control-sequence rules grounded in ASHRAE Guideline 36 [9]. The canonical alphabet has ten activities in five complementary pairs (system, night cycle, heating, cooling and economiser window, each with an entry and an exit event);<sup>1</sup> the single-duct air handler’s configuration yields six of them, the two fan-powered units all ten. Because every activity has an inverse that also occurs many times a day, the traces are loop-heavy — the calibrated healthy band for cooling-active occurrences on the series unit runs from 4 to 44 in a day — and the alphabet is constructed, not emitted: each activity exists because a rule decided that a sustained threshold crossing counts as a state change. That is ordinary for sensor-domain process mining [17], and it is where the analyst’s judgement enters the model.

#### 3.2 The alphabet wall

The events are partitioned into two alphabets that never mix. *State events* are neutral facts about what the equipment did and are the only events admitted to discovery. *Fault-signature events* already encode engineering suspicion — a valve commanded open while the coil it feeds does nothing, a supply temperature contradicting the unit’s claimed mode — and are withheld from discovery, though they remain available to the detection side. The partition exists because of circularity, the process-mining form of leakage [16]: a model discovered over an alphabet that already names faults will later “detect” them, but what it has recognised is the analyst’s own rule, re-encoded as a transition and read back out through alignment cost. We are not aware of a prior quantification on a sensor-derived log. A partition enforced by convention decays, so the separation is held by a tested invariant: injecting fault-signature events into a log must not

<sup>1</sup> `outputs/grammar_results.json`, `canonical_alphabet` and `event_days`.

change any day’s fitness against a discovered net, and the test fails at build time if a future edit lets signature events reach the discovery path.

### 3.3 The bound

Quantifying it needs one comparison, and we state exactly what it is. The unpartitioned number comes from the committed artefact of the pre-partition prototype pipeline; the partitioned number from this pipeline’s artefact on the same single-duct system. The two share the alignment threshold, the 248-day training and 87-day holdout split, the 4,540 scored fault days and — the check that the comparison is not confounded elsewhere — an identical rules-channel output of 3,134 true positives; they do not share a code version, and a controlled same-code re-run is future work. The deployed scorecard, two code generations later, reads the same: 86 of 4,891 evaluable fault days strictly worse than the worst fault-free holdout day (the thresholded channel flags none), 1.76%.<sup>2</sup> Under that comparison the model channel’s apparent recall rises from 1.78% to 5.24% — 81 detected fault days against 238 — when the discovered net may contain the activities that name the faults.<sup>3</sup> The difference, 3.5 percentage points, is a measured bound on model self-confirmation for this system and this alphabet. The two artefacts agree on every combined-detector number — 3,152 true positives and 5 false positives in both — so the partition changed the deployed output by zero days: every detection the leak had supplied was already caught by the rules channel, the redundancy Section 5 measures directly. It is a bound on one system and on inflated recall only; the design is generic.

## 4 Discovery at three strata, and the reversal probe

Petri nets are discovered from the state-event log (Table 1) with the inductive miner [10] in pm4py, at three granularities: a *device* stratum modelling one actuator’s lifecycle with one case per device per day, a *unit* stratum modelling the air handler’s daily cycle, and a cross-system grammar onto the canonical ten activities so that a net discovered on one building can be replayed against another’s log. All discovery uses fault-free data only.

### 4.1 The reversal probe

Whether such a net encodes temporal order, or merely licenses a set of activities in any arrangement, is the question precision metrics [15] answer least reliably on logs with no negative examples [4], which sensor logs never have, so we use a direct probe: reverse every trace, re-score it against the same net, and report the drop in mean fitness. A net that constrains order cannot fit a reversed log

<sup>2</sup> `outputs/benchmark_v6_sdahu.json, model_days_minrobust.`

<sup>3</sup> Partitioned run: `outputs/benchmark_v3.json`; unpartitioned predecessor: `outputs/benchmark_v2_processheal_v1.json`; `structure true` positives 81 and 238 over `pos_days` 4,540.

**Table 1.** The three unit-stratum logs and nets over the fault-free year. “In net” counts the activities the miner kept as labelled transitions; net sizes are places / transitions / silent transitions; fitness is mean alignment fitness on the holdout days ( $n = 87 / 96 / 96$ ), forwards and on reversed traces.

| System               | Act. (in net) | Days | Variants | Net p/t/ $\tau$ | Fitness | Reversed |
|----------------------|---------------|------|----------|-----------------|---------|----------|
| Single-duct AHU      | 6 (6)         | 335  | 102      | 16/17/11        | 0.8861  | 0.5526   |
| Parallel fan-powered | 10 (7)        | 365  | 343      | 15/18/11        | 0.9021  | 0.9026   |
| Series fan-powered   | 10 (8)        | 365  | 364      | 15/17/9         | 0.9533  | 0.9533   |

as well as the original; a net whose fitness is a function of the activity multiset will not notice.<sup>4</sup> The single-duct net falls from 0.8861 to 0.5526: it encodes order. The series fan-powered net scores 0.9533 forwards and 0.9533 backwards, and the parallel net moves from 0.9021 to 0.9026, five ten-thousandths in the wrong direction. Two of the three nets carry no ordering information in their fitness; their high absolute fitness states which activities occurred, not when. Its limit: it tests whether *fitness* responds to order, not whether the net contains order-encoding structure the log never visits.

*The net’s reading added nothing over counting.* A further measurement, computed for the test in Section 6, belongs here. For each system we counted the fault-free occupied days on which a named activity is replayed as a synchronous move against the discovered unit net. On all three systems sealed at the time the count equalled the number of days on which the activity simply appears in the raw log: 0 against 0, 166 against 166, and 357 against 357.<sup>5</sup> At the unit stratum the alignment added nothing that counting would not have given, exactly. The device stratum, pooled across each building’s four terminal units, inverts the unit stratum’s ranking of the two comparable buildings (supports 0.455 and 0.978 against 0.437 and 0.014); a quantity reproducible by counting on one stratum and reversed on another is not yet a measurement.

## 5 What the detector achieves, and the conformance null

Evaluation used public LBNL datasets [8, 7], scored at day level with the scenario as the unit of analysis. The three systems on which the hypothesis was tested — a single-duct air-handling unit and parallel and series fan-powered units — carry 74 seeded fault scenarios of which 73 are scored (one series-unit file has a rotated calendar; the single-duct system’s 20 files hold 14 distinct runs [14]); a dual-duct air handler and a fan coil unit were onboarded afterwards by configuration alone. Eight channels score each day: physics rules, temperature-residual bands, *model conformance* and *device conformance* (alignment cost against the discovered unit and device nets — these two, and only these two, are the conformance channels),

<sup>4</sup> `outputs/grammar_results.json`, `reversal_probe`.

<sup>5</sup> `outputs/discovery_predicts_transfer.json`, Q: `sync_days` against the raw occupied-day count.

*absence* of a scheduled activity on a device, a *frequency* band on the log’s daily activity counts, a band on signal direction changes, and an advisory seasonal rate channel. The absence and frequency channels read the event log the abstraction produced; the two conformance channels rest on the discovered nets. Every channel is calibrated on the fault-free year only and may report only what exceeds its own measured noise floor (a binomial at  $p < 10^{-3}$  against its false-alarm rate on the fault-free holdout days it could evaluate, floored at three days).

### 5.1 The scorecard

Table 2 is the deployed result on all five systems after a pre-registered audit that mapped every recorded column: 158 of 175 seeded scenarios detected at 26 false-alarm days in 480 held out, every threshold out-of-sample, the median time to detection one day. The seventeen misses are coil fouling whose recorded values sit inside every healthy band. The log’s own channels earn their place there. On both fan-powered units the unstable-room-temperature scenario is carried by the frequency channel with the direction-change band and by nothing else, and the unstable-damper scenario by the direction-change band alone; on the dual-duct unit the absence channel is credited on 14 of the 50 detections. Localisation comes from the same log: the device whose signature events fire on most days names the terminal unit, and on the 50 fan-powered detections with determinable ground truth it names the correct unit in 37, none in 13 and a wrong one in none, three wrong units being available in every case.<sup>6</sup> A second pre-registered test maps the carrying rule to the seeded component: exact in 71% of the 158 detections and the right subsystem in 85%, short of the pre-registered component-level bar, so the claim is subsystem-level.<sup>7</sup> Against a repaired open-source Guideline 36 rule battery held to the same silence gate, the detector is within one detection and one false-alarm day on the air handler (13 of 14 at 1 after the adjudication noted in Table 2, against 12 of 14 at 1) and catches three times as many terminal-unit faults (26 of 30 and 25 of 29 at 8 and 9 false alarms against 8 of 30 and 9 of 29 at 0), a coverage difference; the pre-registered claim of at least as many at fewer false alarms fired its falsifier.<sup>8</sup>

### 5.2 The ablation

Because the committed scorecards record, per scenario, which gated channels were credited with the detection, the ablation is a read rather than a re-run (Table 2). No scenario on any system is detected only by the two conformance channels: zero sole detections in 73 scenarios bounds the sole-detection rate below 4.1% (rule of three), and below 1.5% over the 199 scenarios of six systems, the five in the table and a simulated rooftop unit. They are credited with none of

<sup>6</sup> `outputs/alarm_attribution.json;` `outputs/benchmark_v6_*.json,`  
`meaningful_channels` and `top_device`.

<sup>7</sup> `outputs/component_attribution.json, x37_diagnosis.json.`

<sup>8</sup> `outputs/x38_guideline36.json.`

**Table 2.** Deployed scorecard and conformance ablation on the five systems, read from `outputs/benchmark_v6_*.json` (`meaningful_channels`) and `union_fpr_*.json`. FA: fault-free holdout days of 96 flagged by any gated channel (advisory rate channel excluded). “Credited”: detections in which a conformance channel was among the gated channels; “sole”: those in which it was the only one. The single-duct count is the naive 14; one detection is later adjudicated out as branch provenance, which changes no cell.

| System               | Scenarios | Detected | FA / 96 | Credited | Sole |
|----------------------|-----------|----------|---------|----------|------|
| Single-duct AHU      | 14        | 14       | 1       | 0        | 0    |
| Parallel fan-powered | 30        | 26       | 8       | 0        | 0    |
| Series fan-powered   | 29        | 25       | 9       | 7        | 0    |
| Dual-duct AHU        | 55        | 50       | 3       | 3        | 0    |
| Fan coil unit        | 47        | 43       | 5       | 6        | 0    |
| Total                | 175       | 158      | 26      | 16       | 0    |

the 14 single-duct or 26 parallel-unit detections and appear in 7 of the 25 series-unit detections, never alone (the coverage audit left the model and device rows identical); removing both channels leaves every scorecard unchanged. On the series unit the two channels appear on six of the nine false-alarm days of the 96-day fault-free holdout (out-of-sample counts), and deleting them eliminates five (the sixth is also flagged by another channel), so that system’s false-alarm rate falls from 9 in 96 to 4 in 96 at no detection cost; on the parallel unit the same deletion takes 8 in 96 to 6.<sup>9</sup> Their only measurable effect on this detector’s output, then, is those five false alarms.

### 5.3 The matched-rule control

A second control separates what discovery contributes at design time from what it contributes at run time. For each channel a domain expert hand-wrote the rule the channel’s behaviour implied and scored it on the same scenarios. Two reproduce their channel exactly — 141 against 141 on the series unit and 206 against 206 on the parallel unit, both the frequency channel on the unstable-room-temperature scenario: discovery located an invariant nobody had written down, and once written down the channel was exactly redundant with it. The founding hypothesis had a falsifier attached and it fired: on the series unit a one-line rule, *occupied workday and air-handler heating never active*, covered 192 fault days against the model channel’s 135 when the falsifier fired (178 in the current artefact) on the same scenario, with zero false alarms on 365 fault-free days,<sup>10</sup> and the 135 rests four fifths on a threshold interpolated from one holdout day (22–26 under the most conservative). Transplanted, the same rule fires on 231 fault-free days on the single-duct system (which has no hot-water valve, so the rule fires for want of a sensor and says nothing about the models), 124 on

<sup>9</sup> `outputs/union_fpr_sfpu.json`, `channels.*.holdout_fp_dates`: model 5 days, device 1, frequency 1 (shared with the model channel), residual 3; `union_all8` 9 days.

<sup>10</sup> `outputs/matched_rules_sfpu.json`, `SFPU_VAVDMPRStuck_80%`: `mr1` = 192, `model` = 178 (135 at the Phase 3B close), `healthy_fp.mr1` = 0.

the parallel unit and none on the series unit; whether the models predict the 124 against 0 is the subject of Section 6.

#### 5.4 Reading the null

Three readings are available. The conformance channels might be poorly implemented; but they use standard alignment-based replay [1] on nets from a standard miner, and a pre-registered positive control (order violations injected into fault-free holdout traces, scored through the deployed gate) shows what the instrument is: on the fan coil and dual-duct units a reversed day is flagged (55 of 72, 59 of 83), so the channel sees order there and the faults do not change it; on the single-duct unit a reversed day is separable on average (AUC 0.91) but no in-budget threshold sees it; on the fan-powered units fitness does not move.<sup>11</sup> Nor does another instrument do better: a pre-registered grid of seven (inductive miner at noise 0.0 and 0.5, token replay, heuristics miner, log skeleton, Declare, temporal profile), each discovered and thresholded as the deployed channel is, detects none of the 33 scenarios then missed on four systems (realised false-alarm rates 0 to 9.7%; sixteen were later recovered by the coverage audit); two heuristics-miner cells timed out and are reported as not evaluated.<sup>12</sup> The domain might be wrong for the method; partly, since two of the three nets do not encode order. The third reading: every genuine model-channel detection we inspected was a *count* deviation — extra heating episodes, collapsed fan cycles, a vanished daily rhythm — surfacing through alignment cost, and once a frequency channel measured count deviations directly and two-sidedly, the conformance channels had nothing uniquely theirs. Alignment cost was acting as a noisy, one-sided, expensive frequency detector; if that generalises, conformance-as-anomaly-detection will underperform wherever the net is order-permissive, which is wherever the alphabet is loop-heavy, as sensor-derived alphabets of complementary pairs cycling many times a day are. Checking the log rather than the model: for eight of the twelve scenarios then missed, all reheat-coil fouling, the state-event log is indistinguishable from healthy at 5% in every daily activity count and state duration, so no method reading that log can detect them at the deployed alphabet; under valve faults it changes in timing and counts, not order. A pre-registered time-infused state model is significant on 11 of 73 scenarios, none the deployed channels miss, at five extra false-alarm days per fan-powered system; enriching the alphabet with healthy-derived actuator and flow band states catches one missed scenario at 18 and 21 false-alarm days in 96, over budget, and a frequency channel on that alphabet keeps the budget but catches none of the fourth system’s ten misses, so it is not adopted.<sup>13</sup> The null also transfers: the dual-duct and fan coil rows of Table 2 were onboarded blind by configuration

<sup>11</sup> `outputs/x22_positive_control.json`.

<sup>12</sup> `outputs/x30_method_grid.json`.

<sup>13</sup> `outputs/x12_log_diagnosis.json`, `x12_time_perspective.json`,  
`x14_enriched_alphabet.json`, `x14_control.json`, `x15_enriched_frequency.json`;  
each records its pre-registration and amendments.



alone (45 of 55 at 3 and 40 of 47 at 4 before the coverage audit), the conformance channels credited only on scenarios other channels already carry. The fan coil unit’s first run had 25 weekend model-channel false-alarm days of 28 and six model-only “detections” on those weekends; the pre-registered repair moved no earlier scorecard.<sup>14</sup>

## 6 A pre-registered test that never had power

One claim for the discovered models remained after the null: that they *locate* invariants, so that a rule written from a model’s reading of one building holds on another building where that model’s support for the invariant is high. Three hand-written rules had been transplanted across the three buildings with their fault-free firings recorded, and a test was pre-registered: the model-derived quantity (the fraction of fault-free occupied days whose optimal alignment to the discovered unit net contains a synchronous move on the heating activity), three predictions and three falsifiers were committed before computation, and the quantity was computed by an agent instructed not to open the rule-firing artefacts.<sup>15</sup>

The test failed for four reasons, each caught later than it should have been. The scorer was changed once after registration (Monte Carlo to exact permutation  $p$ , verdict-neutral, but without a dated amendment). The pre-registration’s disclosure section printed the rule’s firings, unblinding the two informative cells. An amendment made before computation excluded the single-duct system for the correct reason given in Section 5, and a second, after the fact, blamed the resulting six-cell design for the lack of power, since two groups of three admit a minimum exact  $p$  of 0.05 against a criterion of  $p < 0.05$ . That diagnosis was itself wrong. The model quantity is a property of a building’s net and is constant across rules within a building (0.000, 0.455 and 0.978); the nine cells were three buildings pseudo-replicated, the valid permutation null is over buildings, and with three the minimum attainable one-sided  $p$  is  $1/3! = 0.167$ ; with the two the amended test actually scored it is  $1/2! = 0.5$ . The test never had power at any stage; the reported  $p = 0.20$  is not a measurement, and this paper draws no inference about the claim in either direction.

The lesson is the contribution. Pre-registration fixed the quantity, the predictions and the falsifiers; it did not fix the unit of analysis, and two amendments under the same discipline did not either. The remedy is a mandatory pre-registration field naming the unit of replication and the minimum attainable  $p$  under a permutation of that unit, computed before any prediction is written. With four scorable buildings the re-run had power and passed ( $\rho = -1$ , exact  $p = 1/24$ ); but the model quantity equals the log’s own heating-day count on three of the four buildings and the transplanted rule is a heating-absence rule, so

<sup>14</sup> `outputs/x17_onboarding.json`, `x19_onboarding_run1.json`, `x19_onboarding.json`.

<sup>15</sup> `docs/plans/2026-09-23-discovery-predicts-transfer-prereg.md`, with three dated amendments.

a discovery-free control added after the result — that count — passes identically and the pass supports nothing about discovery. A net fitted on one building and read against another’s year is not that count, and this first differing ordering runs the wrong way at  $n = 4$ .<sup>16</sup>

## 7 Tests that pin refutations and disclosures

Every result here is guarded by an automated test that runs on every commit. Beyond regenerating results, it pins three things we have not found guarded elsewhere. It pins *fired falsifiers*: the sampling-rate experiment’s finding is its four refutations, and a regeneration that lost one fails the build as a lost feature would. It pins *disclosure obligations*: that the false-alarm artefact still declares each channel’s threshold provenance, and that the transfer artefact still states its design’s floor. And it pins *provenance*: the pre-registration, its first amendment and the scorer must be git ancestors of the commit that first added the result. Caveats decay faster than code, so a lesson is not closed until a test enforces it.

## 8 Limitations

Every detection result comes from simulation (the deployed budget has since been measured at 6.1 % of held-out days on one real rooftop unit), and simulation flatters: noise-free years make thresholds cheap and silence easy. For the null the bias runs the other way — a clean training year should, if anything, help a discovered net — but real, noisier logs could change the relative standing of conformance against the statistical channels. Two pre-registered injections (sensor jitter; field sensor error, change-of-value logging and schedule shifts) left detection counts within three of clean and false alarms at or below 4 in 96, and the second, repeated on the post-coverage configurations of all five systems, cost at most one detection per system and condition at worst 7 false alarms in 96; on the one LBNL dataset that records the refrigerant side (a simulated rooftop unit) the unchanged detector catches all ten seeded coil-fouling severities at no false alarm in 28 held-out days.<sup>17</sup> Three buildings at the time of the transfer test is the harder limit: a building is the unit across which a discovered model must transfer, two of the three are variants of one equipment family, and all share a simulated weather-year family. The noise-floor gate runs one binomial test per scenario and channel, several hundred at  $\alpha = 10^{-3}$ , with no multiplicity correction; per-day flags are autocorrelated through weather, so every  $p$  is optimistic; and the conformance thresholds, though calibrated on a separate slice, come from the same fault-free year as every band. The coverage audit chose its columns after a read-only scan of the fault files, so its sixteen recoveries show the

<sup>16</sup> `outputs/x20_transfer_four.json,` `x21_foreign_net_support.json;`  
`docs/plans/2026-09-24-x20-transfer-four-buildings-prereg.md.`

<sup>17</sup> `outputs/x9_x10_robustness.json,` `x31_field_conditions.json,`  
`x35_field_conditions.json, x29_refrigerant_observability.json.`

misses were reachable from the recorded columns with blind thresholds, not that a blind onboarding would have reached them. Eight machine-verified benchmark defects are reported in a companion paper [14]. AI coding agents were used for implementation, analysis and adversarial recomputation (Section 6 reports how far a sealed computation held); all designs, pre-registrations and claims were authored and approved by the author.

## 9 Conclusion

We carried event-log process mining into a domain it had not been applied to and built a detector that works: 158 of 175 seeded faults on five public systems at 26 false-alarm days in 480, the terminal unit named and never misnamed, the budget held under field conditions. Process mining is in that result in three places: the event abstraction from the control sequence is the log every channel reads; the stratified structure gives the log a per-device case from which localisation and the absence channel follow; and the frequency channel on the log carries detections no rule or residual carries. Discovery also earned its place at design time, by locating an invariant an expert could then write down. What process mining does not do here is detect on its own: the pre-registered hypothesis that conformance against the discovered nets would catch the faults had a falsifier attached, the falsifier fired, and the ablation, the circularity bound, the reversal probe and the alignment-support count explain why — the nets are order-permissive on a loop-heavy alphabet, and what alignment cost sees, a count sees first. Those three measurements are cheap, none is specific to buildings, and each belongs before any deployment of conformance as an anomaly detector. Process mining helps to get detection; it is not detecting on its own.

*Acknowledgements.* The author thanks Anthony Aighobahi and Mridula Sharma for their supervision. The use of AI coding assistants is stated in Section 8.

## References

1. van der Aalst, W.M.P., Adriansyah, A., van Dongen, B.: Replaying history on process models for conformance checking and performance analysis. *WIREs Data Mining and Knowledge Discovery* **2**(2), 182–192 (2012). <https://doi.org/10.1002/widm.1045>
2. Bezerra, F., Wainer, J.: Algorithms for anomaly detection of traces in logs of process aware information systems. *Information Systems* **38**(1), 33–44 (2013). <https://doi.org/10.1016/j.is.2012.04.004>
3. Bi, J., Wang, H., Yan, E., Wang, C., Yan, K., Jiang, L., Yang, B.: AI in HVAC fault detection and diagnosis: A systematic review. *Energy Reviews* **3**(2), 100071 (2024). <https://doi.org/10.1016/j.enrev.2024.100071>
4. vanden Broucke, S.K.L.M., De Weerd, J., Vanthienen, J., Baesens, B.: Determining process model precision and generalization with weighted artificial negative events. *IEEE Transactions on Knowledge and Data Engineering* **26**(8), 1877–1889 (2014). <https://doi.org/10.1109/TKDE.2013.130>

5. Brzychczy, E., Aleknytytė-Resch, M., Janssen, D., Koschmider, A.: Process mining on sensor data: a review of related works. *Knowledge and Information Systems* **67**, 4915–4948 (2025). <https://doi.org/10.1007/s10115-024-02297-y>
6. Crowe, E., Chen, Y., Reeve, H., Yuill, D., Ebrahimifakhar, A., Chen, Y., Troup, L., Smith, A., Granderson, J.: Empirical analysis of the prevalence of HVAC faults in commercial buildings. *Science and Technology for the Built Environment* **29**(10), 1042–1057 (2023). <https://doi.org/10.1080/23744731.2023.2263324>
7. Granderson, J., Lin, G., Chen, Y., Casillas, A., Wen, J., Chen, Z., Im, P., Huang, S., Ling, J.: A labeled dataset for building HVAC systems operating in faulted and fault-free states. *Scientific Data* **10**, 342 (2023). <https://doi.org/10.1038/s41597-023-02197-w>
8. Granderson, J., Lin, G., Harding, A., Im, P., Chen, Y.: Building fault detection data to aid diagnostic algorithm creation and performance testing. *Scientific Data* **7**, 65 (2020). <https://doi.org/10.1038/s41597-020-0398-6>
9. Janssen, D., Mannhardt, F., Koschmider, A., van Zelst, S.J.: Process model discovery from sensor event data. In: *Process Mining Workshops (ICPM 2020)*. LNBIP, vol. 406, pp. 69–81. Springer (2021). [https://doi.org/10.1007/978-3-030-72693-5\\_6](https://doi.org/10.1007/978-3-030-72693-5_6)
10. Leemans, S.J.J., Fahland, D., van der Aalst, W.M.P.: Discovering block-structured process models from event logs — a constructive approach. In: *Application and Theory of Petri Nets and Concurrency (PETRI NETS 2013)*. Lecture Notes in Computer Science, vol. 7927, pp. 311–329. Springer (2013). [https://doi.org/10.1007/978-3-642-38697-8\\_17](https://doi.org/10.1007/978-3-642-38697-8_17)
11. Nolle, T., Luetzgen, S., Seeliger, A., Mühlhäuser, M.: BINet: Multi-perspective business process anomaly classification. *Information Systems* **103**, 101458 (2022). <https://doi.org/10.1016/j.is.2019.101458>
12. Pitsch, C., Brockhoff, T., Adams, J.N., Leemans, S.J.J., Celi, L.A., van der Aalst, W.M.P.: Hypothesis testing for processes. In: *2025 7th International Conference on Process Mining (ICPM)*. pp. 1–8. IEEE (2025). <https://doi.org/10.1109/ICPM66919.2025.11220677>
13. Rozinat, A., van der Aalst, W.M.P.: Conformance checking of processes based on monitoring real behavior. *Information Systems* **33**(1), 64–95 (2008). <https://doi.org/10.1016/j.is.2007.07.001>
14. Singh, Y.: Five defects in the LBNL HVAC fault-detection benchmarks, and the gate battery that found them (2027), companion paper, submitted to the R2PM workshop at ICPM 2027; in preparation
15. Tax, N., Lu, X., Sidorova, N., Fahland, D., van der Aalst, W.M.P.: The imprecisions of precision measures in process mining. *Information Processing Letters* **135**, 1–8 (2018). <https://doi.org/10.1016/j.ipl.2018.01.013>
16. Weytjens, H., De Weerd, J.: Creating unbiased public benchmark datasets with data leakage prevention for predictive process monitoring. In: *Business Process Management Workshops (BPM 2021)*. Lecture Notes in Business Information Processing, vol. 436, pp. 18–29. Springer (2022). [https://doi.org/10.1007/978-3-030-94343-1\\_2](https://doi.org/10.1007/978-3-030-94343-1_2)
17. van Zelst, S.J., Mannhardt, F., de Leoni, M., Koschmider, A.: Event abstraction in process mining: literature review and taxonomy. *Granular Computing* **6**(3), 719–736 (2021). <https://doi.org/10.1007/s41066-020-00226-2>
18. Zhong, Y., Lisitsa, A.: Can process mining help in anomaly-based intrusion detection? arXiv:2206.10379 (2022), preprint; not peer reviewed. Reports that naïve process mining on network event logs was ineffective for intrusion detection